YAMLess Case Study — TryGhost/Ghost

YAMLess: Generating Azure Infrastructure from Code

Case Study — TryGhost/Ghost

What is YAMLess?

YAMLess is an **Infrastructure from Code (IFC)** tool. Given a code repository, it produces a complete, policy-validated Azure Terraform workspace — with no configuration files, no schema to fill in, and no IaC expertise required from the developer.

The pipeline has four stages:

```
Repo on disk
    │
    ▼
┌─────────────────────┐
│  1. Static Analyzer │  Reads Dockerfile, package.json / requirements.txt,
│                     │  .env files, source code. No execution, no API calls.
│                     │  Produces a typed AppProfile with confidence scores.
└──────────┬──────────┘
           │  (fires only when any field is low/medium confidence)
           ▼
┌─────────────────────┐
│  2. LLM Gap-Filler  │  Claude reads relevant source files and fills in what
│                     │  static analysis couldn't determine — via structured
│                     │  tool use only, no free-form parsing.
└──────────┬──────────┘
           │
           ▼
┌─────────────────────┐
│  3. Renderer        │  Jinja2 root templates + static Terraform modules →
│                     │  ready-to-apply workspace. Outputs Container Apps
│                     │  or AKS, selected by --target.
└──────────┬──────────┘
           │
```

```
          ▼
┌──────────────────────┐
│  4. Validator        │  checkov policy-as-code. Security posture is enforced
│                      │  by static module files — the LLM cannot affect it.
└──────────────────────┘
```

Applying YAMLess to TryGhost/Ghost

Ghost is a widely-used open-source publishing platform: Node.js, MySQL or SQLite, optional S3-compatible storage, Stripe payments, and a rich plugin ecosystem. It is structured as a **pnpm monorepo** with the core application at `ghost/` rather than the repo root.

Command run

```
yamless generate /tmp/ghost \
  --no-llm \
  --out-dir ~/yamless-outputs/ghost-infra \
  --validate
```

Profile detected

Field	Detected value	Confidence	Notes
Runtime	`node`	medium	Detected from Dockerfile, but multi-stage build reduces confidence
Runtime version	`null`	—	Multi-stage Dockerfile; runtime FROM is not the first stage
Port	`2368`	high	`EXPOSE 2368` in Dockerfile
Shape	`http-service`	medium	HTTP framework not visible at root; port presence used as signal

Field	Detected value	Confidence	Notes
Database	`null`	—	`mysql2` is in `ghost/package.json`, not the root manifest
Storage	`false`	—	`@aws-sdk/client-s3` is in `ghost/package.json`, not root
Cache	`false`	—	Not detected
Ingress	`external: true`, port `2368`	high	
Secrets	`GITHUB_TOKEN`, `STRIPE_SECRET_KEY`	high	Only 2 of ~15 real secrets detected

Output generated

```
ghost-infra/
├── main.tf              # Identity, KV, Container App Env, Container App
├── variables.tf
├── outputs.tf
├── providers.tf
└── modules/
    ├── managed_identity/
    ├── keyvault/
    ├── container_app_env/
    └── container_app/      # No database or storage — not detected
```

22/22 policy checks passed — the output is valid and deployable, but incomplete: no database and no storage module were generated because neither was detected from the root package manifest.

What the output correctly captured

- External HTTP ingress on port 2368
- Managed Identity with Key Vault wiring
- Stripe and GitHub secrets surfaced and provisioned in Key Vault
- Correct scaling profile (HTTP trigger, 1–10 replicas)
- All security golden standards: RBAC-only auth, TLS 1.2, KV firewall deny-all, consistent tagging

What was missed

Missing	Root cause
MySQL database module	`mysql2` lives in `ghost/package.json`; analyzer only reads root
Storage module	`@aws-sdk/client-s3` lives in `ghost/package.json`; same issue
13 additional secrets	Ghost's env vars are in nested config files, not a root `.env`
Runtime version	Multi-stage Dockerfile — the `node:xx` base is in a build-only stage

Learnings for YAMLess

1. Monorepo support is a hard requirement

What happened: Ghost uses a pnpm workspace. The root `package.json` is the workspace manifest — it declares workspaces but carries no application dependencies. The actual app lives at `ghost/package.json`, where `mysql2`, `@aws-sdk/client-s3`, `bookshelf`, and `knex` are all present.

YAMLess reads only the root `package.json` and found nothing, resulting in `database: null` and `storage: false` — both silently wrong.

Proposed fix: Before reading `package.json`, detect monorepo indicators:

Indicator	Monorepo type
`pnpm-workspace.yaml`	pnpm
`nx.json`	Nx
`lerna.json`	Lerna
`workspaces` field in root `package.json`	Yarn/npm workspaces
`packages/` or `apps/` directory with nested `package.json`	Generic

When a monorepo is detected, walk inward to find the package that matches the app name (or the deepest single candidate), and run dependency analysis against that package.

Impact: This single change would have correctly detected MySQL and S3 storage for Ghost.

---

2. Multi-stage Dockerfile parsing is needed for accurate runtime detection

What happened: Ghost's Dockerfile uses named build stages. The first `FROM` is a build stage; the actual runtime stage (`FROM node:xx-alpine`) comes later. YAMLess reads only the first `FROM` and the first `EXPOSE`, which gives a `medium` -confidence runtime and a missing version.

Proposed fix: Parse all `FROM` directives, identify the final stage (the one not referenced by any other `AS <stage>` target), and read runtime and version from that stage.

---

3. MySQL/MariaDB needs a renderer module

What happened: Even if the analyzer had correctly detected `mysql2` , the renderer has no MySQL module. The profile would carry `database: mysql` and silently skip the database module block.

Proposed fix: Add an Azure Database for MySQL Flexible Server module to both the Container Apps and AKS renderers, mirroring the existing Postgres module.

---

4. Secret scanning should recurse into monorepo packages

What happened: Ghost has ~15 meaningful environment variables (`database__connection__password` , `mail__transport` , `STRIPE_WEBHOOK_SECRET` , etc.) spread across nested config files and `.env.example` files. Only 2 were found because the env scanner looks at the repo root only.

Proposed fix: When a monorepo is detected, extend env scanning into subdirectories and add a parser for dot-notation config keys (`database__connection__host`) used by Ghost's config library.

Improvement Roadmap

Priority	Change	Impact
High	Monorepo detection + inner package.json resolution	Fixes dependency and secret gaps for Ghost, Redash, Plane, and most modern JS repos
High	MySQL/MariaDB Terraform module	Unblocks a large class of apps that don't use Postgres
Medium	Multi-stage Dockerfile parser	Accurate runtime version for all containerised apps
Medium	Recursive secret scanning with dot-notation config support	More complete secret inventory for Node apps using config libraries
Low	Redis/cache Terraform module (Azure Cache for Redis)	Detected in Outline and Ghost but not provisioned

---

Summary

Ghost is a valuable test case precisely because it breaks assumptions that hold for simpler repos. YAMLess produced valid, policy-passing Terraform — but the output is incomplete due to the monorepo structure hiding the real dependency manifest. The tool's confidence scoring worked correctly (medium confidence on runtime and shape correctly triggered the LLM gap-filler flag), but the gap-filler cannot compensate for data that was never collected.

The fix is upstream in the static analyzer: teach it to recognise monorepo layouts and walk to the right package before classifying dependencies.

Generated by YAMLess — https://github.com/zafar-ahmed-kr/YAMLess